Chapter 5

Introduction to Machine Learning
Foundations of Data Science, 2nd Edition

5.1 What is machine learning

Machine learning is a branch of artificial intelligence where a system learns patterns from data instead of following hand-written rules. In traditional programming, a developer writes the rules and the program applies them to data. In machine learning, the developer provides data and the system finds the rules.

A spam filter is a simple example. A traditional filter might block every message that contains the word "lottery". A machine learning filter learns from thousands of labelled emails which combinations of words, links, and sender details are likely to be spam. It can then classify new emails it has never seen before.

Machine learning works best when the problem has many examples, the pattern is stable, and the outcome can be measured. It is a poor fit for problems with very little data or rules that change every day.

5.2 Types of learning

Supervised learning

Supervised learning uses labelled data. Each training example has an input and a known output. The model learns to map inputs to outputs. When the output is a category, the task is classification, such as marking an email as spam or not spam. When the output is a number, the task is regression, such as predicting a house price.

Common supervised algorithms include linear regression, logistic regression, decision trees, random forests, and support vector machines.

Unsupervised learning

Unsupervised learning uses data without labels. The model looks for structure in the data. Clustering groups similar items, for example grouping customers by buying habits. Dimensionality reduction compresses many features into a smaller set while keeping the important variation. K-means is a common clustering method, and PCA (principal component analysis) is a common dimensionality reduction method.

Semi-supervised learning

Semi-supervised learning combines a small labelled set with a large unlabelled set. It is useful when labelling is expensive but raw data is easy to collect.

Reinforcement learning

Reinforcement learning trains an agent to make decisions in an environment. The agent takes actions, receives rewards or penalties, and learns a policy that maximises the total reward over time. Game playing and robot control are common examples.

5.3 The training process

A typical machine learning project follows a sequence of steps. The quality of the data usually matters more than the choice of algorithm.

Step 1: Collect and clean the data

Collect examples that match the real problem. Remove duplicates, fix missing values, and correct obvious errors. A common rule is that 80 percent of the effort goes into data preparation.

Step 2: Engineer features

A feature is a measurable input property. Feature engineering turns raw data into useful numbers, such as converting a date into day of week or scaling a price into a range between 0 and 1.

Step 3: Split the data

Split the data into a training set, a validation set, and a test set. A common split is 70 percent training, 15 percent validation, and 15 percent test. The model learns on the training set, tuning uses the validation set, and the final score comes from the test set. The test set must never be used during training.

Step 4: Train the model

Training adjusts the model parameters to reduce a loss function. The optimiser updates the parameters in small steps. An epoch is one full pass over the training data. The batch size is the number of examples used in one update. The learning rate controls the size of each step. A learning rate that is too high can overshoot, and one that is too low makes training very slow.

Step 5: Evaluate and tune

After training, measure the model on the validation set. Tune hyperparameters such as learning rate, batch size, and tree depth. Hyperparameters are set before training, while parameters are learned during training.

5.4 Overfitting and underfitting

Overfitting happens when a model learns the training data too closely, including its noise. It scores well on training data but poorly on new data. Underfitting happens when a model is too simple to capture the real pattern, so it scores poorly on both training and test data.

Ways to reduce overfitting include collecting more data, simplifying the model, adding regularisation, and using early stopping. Ways to reduce underfitting include using a more powerful model, adding better features, and training longer.

The bias-variance tradeoff describes the balance. A high-bias model is too simple and underfits. A high-variance model is too sensitive to the training data and overfits. The goal is a model with low bias and low variance.

5.5 Evaluating a model

Accuracy is the share of predictions that are correct. It is easy to understand but misleading when the classes are unbalanced. If 95 percent of emails are not spam, a model that always predicts not spam has 95 percent accuracy and still misses every spam email.

Precision, recall, and F1

Precision answers: of the items the model marked positive, how many were actually positive? Recall answers: of the items that were actually positive, how many did the model find? The formulas use true positives (TP), false positives (FP), and false negatives (FN).

    Precision = TP / (TP + FP)

    Recall = TP / (TP + FN)

    F1 = 2 * (precision * recall) / (precision + recall)

The F1 score is the harmonic mean of precision and recall. It is useful when you need a single number and the classes are unbalanced.

Confusion matrix and cross-validation

A confusion matrix lists true positives, false positives, true negatives, and false negatives. Cross-validation splits the data into k folds, trains k times, and averages the scores. Common values are k = 5 and k = 10. Cross-validation gives a more stable estimate than a single split.

5.6 Applications

Image recognition: labelling objects, faces, and medical scans.

Natural language processing: translation, summarisation, and chatbots.

Recommendation systems: suggesting products, films, or music.

Fraud detection: flagging unusual transactions in real time.

Healthcare: predicting risk and helping radiologists prioritise scans.

5.7 Challenges

Data quality is the first challenge. Biased or incomplete data produces biased or unreliable models. Interpretability is the second. Some models, such as deep neural networks, are hard to explain, which matters in medicine and finance. Compute cost is the third. Large models need expensive hardware and energy. Privacy is the fourth. Models trained on personal data must follow privacy rules and protect sensitive records.

5.8 Key terms

Model: the learned function that maps inputs to outputs.

Feature: a measurable input property.

Label: the known output for a training example.

Training set: the data used to fit the model.

Validation set: the data used to tune the model.

Test set: the data used for the final score.

Hyperparameter: a setting chosen before training.

Overfitting: learning the training data too closely.

This chapter is a sample created for a demo. It is not a substitute for a course.